INSTITUTE OF ENGINEERING & MANAGEMENT, KOLKATA

COURSE TITLE: BACHELOR OF TECHNOLOGY

SUBJECT: PYTHON PROGRAMMING LABORATORY

PROJECT TITLE: INVENTORY AND REORDER TRACKER - CONSOLE-BASED APPLICATION

No.	Name	Enrollment No.	Roll No.
1	Arnab Satpati	12025052003148	136
2	Chandrima Palit	12025052003149	137
3	Saurav Suman	12025052003150	138
4	Dipannita Dey	12025052003151	139
5	Sinjini Chowdhury	12025052003152	140

SECTION: B

SUBMITTED TO: PYTHON LABORATORY

DEPARTMENT NAME: ELECTRONICS AND COMMUNICATION ENGINEERING

ACADEMIC SESSION: 2026-2027

INVENTORY AND REORDER TRACKER

Abstract

This project develops a Python console application that helps a small shop or store room keep track of products, stock movements and reorder needs. Products are stored with a unique ID, name, category, unit price, current quantity and reorder level. Every stock-in and stock-out is recorded with a timestamp, price and value. The program validates all input, raises **normal** and **urgent** reorder alerts using documented thresholds, calculates stock valuation, and produces NumPy summaries, Pandas category and monthly reports, Matplotlib charts and JSON exports. Data persists in CSV files that are written safely to prevent accidental loss.

The built-in demonstration dataset contains 12 products holding 303 units worth Rs 141,030.00 (Rs = Indian rupees), of which 7 products raise alerts (2 urgent). The project is verified by 51 automated `unittest` tests, all of which passed when this report was prepared. The demonstration data is synthetic and is not taken from a real business. The estimated sales-value chart is an estimate and not audited revenue.

Keywords

Inventory management; reorder alert; stock valuation; Python; CSV; NumPy; Pandas; Matplotlib; JSON; unittest.

Table of Contents

1. Introduction
2. Problem Statement and Objectives
3. Software and System Requirements
4. System Design and Methodology
5. Implementation of Major Modules
6. Results and Discussion
7. Testing and Validation
8. Limitations and Future Scope
9. Conclusion
10. References
11. Appendix: Data Formats and Execution

1. Introduction

Small businesses, college laboratories and stores often track stock in notebooks or loosely organised spreadsheets. Mistakes such as selling more than is available, forgetting to reorder, or losing data are common. A small, reliable program can enforce validation rules and warn the user before an item runs out.

The Inventory and Reorder Tracker is a menu-driven console program. It needs no web server or graphical toolkit, which keeps it easy to run, test and explain. It is organised into small classes and functions so that each responsibility (validation, stock rules, storage, analysis, display) can be understood and tested separately.

2. Problem Statement and Objectives

Problem statement: Build a console application that stores products and stock transactions in CSV files, prevents invalid stock operations, warns about low stock, and summarises inventory using NumPy, Pandas and Matplotlib, while remaining modular, documented and automatically tested.

The principal objectives are:

- Add, view, search and update products without accidentally changing stock.
- Perform stock-in and stock-out, rejecting stock-out quantities above available stock, and log every transaction with a unique ID.
- Display NORMAL and URGENT reorder alerts using clearly defined thresholds.
- Calculate individual and total stock value with correct currency rounding.
- Provide a NumPy statistical summary, a Pandas category report and a monthly in/out report.
- Draw stock-by-product and estimated stock-out-value charts, and export data as valid JSON.
- Persist data in CSV with graceful handling of missing files, malformed rows and permission errors.
- Verify the behaviour with automated tests that never touch the user's real data.

3. Software and System Requirements

Component	Purpose
Python 3.10 or newer	Core program execution; dataclasses, type hints
csv, json, decimal, pathlib, argparse	Persistence, export, exact currency arithmetic, file handling
NumPy	Array statistics (sum, mean, median, minimum, maximum)
Pandas	Group-by reports, pivot tables, date handling
Matplotlib	Bar charts saved as PNG (optionally displayed)
unittest (built in)	Automated testing
Terminal / command prompt	Console user interface (Unicode and colour supported but optional)

Any desktop operating system (Windows, macOS, Linux) with Python and about 100 MB of free disk space is sufficient. Third-party packages are installed with `pip install -r requirements.txt`.

4. System Design and Methodology

4.1 Layered architecture

The application is split into layers. A layer only calls the layers below it, which keeps the business rules independent of the screen and the files.

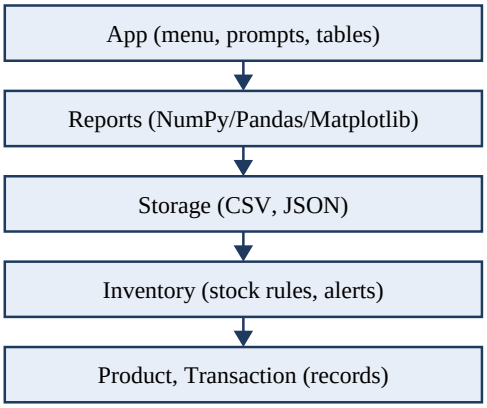


Figure 1. Layers of the application.

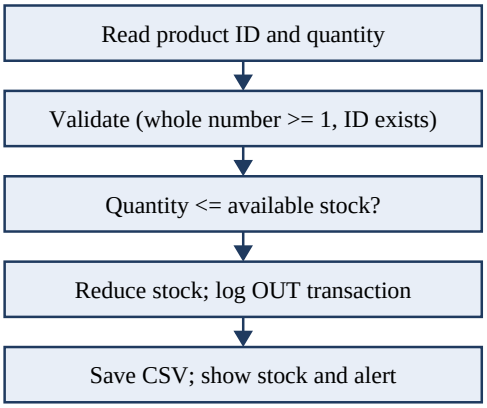


Figure 2. Stock-out flow (rejected if quantity exceeds stock).

4.2 Data schema

File	Columns
products.csv	product_id, name, category, unit_price, quantity, reorder_level
transactions.csv	transaction_id (T000001...), timestamp (YYYY-MM-DD HH:MM:SS), product_id, type (IN or OUT), quantity, unit_price, value

4.3 Validation rules

- Text fields are trimmed and cannot be blank; product IDs are upper-cased and may contain only letters, digits, '-' and '_'.
- Quantities must be whole numbers; prices must be finite, non-negative numbers rounded half-up to two decimals.
- Stock-in and stock-out quantities must be at least 1. Duplicate and unknown product IDs are rejected.
- Validation happens *before* any change, so a rejected action leaves stock and the transaction log untouched.

4.4 Reorder alert thresholds

Level	Condition	Example (reorder level 10)
No alert	quantity above reorder level	11 units
NORMAL	quantity <= reorder level	10 units, 6 units
URGENT	quantity is 0, or quantity <= 50% of reorder level (quantity x 2 <= reorder level)	5 units, 0 units

4.5 Safe saving

Both CSV files are first written to temporary files. Only when both succeed is each original copied to a `.bak` backup and replaced atomically. When loading, malformed rows are skipped with a warning and the original files are copied to `*.corrupt_<time>` so that a later save cannot destroy them.

5. Implementation of Major Modules

Module / function	Functionality
clean_text(), clean_id(), parse_quantity(), parse_price(), line_value()	Input validation, normalisation and exact currency rounding (Decimal)
Product, Transaction (dataclasses)	Validated records; Product validates itself on creation
Inventory.add_product(), update_product()	Create and edit products; update cannot change quantity
Inventory.stock_in(), stock_out()	Change stock, reject insufficient stock, log transactions
alert_level(), Inventory.alerts()	NORMAL / URGENT classification, most urgent first
Inventory.search(), total_value(), seed_demo()	Search by ID/name/category, valuation, demo data without overwriting
Storage.load(), save(), export_json()	Safe CSV persistence and JSON export
numpy_summary()	Total units, mean, median, minimum, maximum, total value
category_report(), monthly_report(), daily_stock_out_value()	Pandas group-by and pivot analysis
stock_chart(), sales_chart()	Matplotlib charts; return None for empty data
App (run, add_product, stock_out, ...)	Menu, prompts, formatted tables, error messages

5.1 Key code

The alert rule and the stock-out operation are shown below exactly as implemented.

```
def alert_level(quantity: int, reorder_level: int) -> Optional[str]:
    """Return URGENT, NORMAL or None for a stock position (thresholds in module doc)."""
    if quantity == 0 or quantity * 2 <= reorder_level:
        return URGENT
    if quantity <= reorder_level:
        return NORMAL
    return None
```

```
    def stock_out(self, product_id: str, quantity: object) -> Transaction:
        product = self.get_product(product_id)
        qty = parse_quantity(quantity, "Quantity", 1)
        if qty > product.quantity:
            raise InsufficientStockError(
                f"Only {product.quantity} unit(s) of {product.product_id} available; "
                f"cannot remove {qty}.")
        product.quantity -= qty
        return self._record(product, STOCK_OUT, qty)
```

5.2 Use of NumPy and Pandas

NumPy: `numpy_summary()` converts quantities and stock values into arrays and uses `sum`, `mean`, `median`, `min` and `max`. **Pandas:** the category report uses `groupby().agg()`; the monthly report converts timestamps to dates and uses `pivot_table()` to separate stock-in and stock-out; the sales chart groups stock-out value by date.

5.3 Menu

Group	Options
Inventory	1 Add product, 2 View inventory, 3 Search, 4 Update
Stock	5 Stock in, 6 Stock out, 7 Reorder alerts
Analytics	8 Valuation, 9 NumPy summary, 10 Category report, 11 Transaction history, 12 Monthly report
Charts and data	13 Stock chart, 14 Estimated sales-value chart, 15 Export JSON, 16 Load demo data
Exit	0 Save and exit

6. Results and Discussion

The following results were produced by loading the built-in demonstration data into an empty inventory and running the report functions of the submitted program. The data is synthetic; month labels depend on the date the demo data is loaded.

6.1 Summary indicators

Indicator	Observed value
Products	12
Total units	303
Mean / median units per product	25.25 / 24.0
Minimum / maximum units	0 / 55
Total inventory value	Rs 141,030.00
Products with alerts	7 (2 urgent, 5 normal)

6.2 Inventory table

```
  Inventory
+------+---------------------+-------------+-------------+-----+---------+--------------+--------+
| ID   | Name                | Category    |       Price | Qty | Reorder |        Value | Alert  |
+------+---------------------+-------------+-------------+-----+---------+--------------+--------+
| P001 | Wireless Mouse      | Electronics |   Rs 799.00 |  23 |      20 | Rs 18,377.00 | -      |
| P002 | USB-C Cable         | Electronics |   Rs 249.00 |  25 |      40 |  Rs 6,225.00 | NORMAL |
| P003 | Mechanical Keyboard | Electronics | Rs 2,499.00 |   5 |      10 | Rs 12,495.00 | URGENT |
| P004 | A4 Notebook         | Stationery  |    Rs 60.00 |  50 |      50 |  Rs 3,000.00 | NORMAL |
| P005 | Gel Pen (Box of 10) | Stationery  |   Rs 120.00 |  45 |      25 |  Rs 5,400.00 | -      |
| P006 | Stapler             | Stationery  |   Rs 150.00 |   8 |       8 |  Rs 1,200.00 | NORMAL |
| P007 | Office Chair        | Furniture   | Rs 5,499.00 |   5 |       5 | Rs 27,495.00 | NORMAL |
| P008 | Desk Lamp           | Furniture   |   Rs 899.00 |   0 |       6 |      Rs 0.00 | URGENT |
| P009 | Water Bottle        | Home        |   Rs 349.00 |  35 |      20 | Rs 12,215.00 | -      |
| P010 | Laptop Stand        | Electronics | Rs 1,299.00 |  30 |      12 | Rs 38,970.00 | -      |
| P011 | Whiteboard Marker   | Stationery  |    Rs 45.00 |  55 |      60 |  Rs 2,475.00 | NORMAL |
| P012 | Extension Board     | Electronics |   Rs 599.00 |  22 |      10 | Rs 13,178.00 | -      |
+------+---------------------+-------------+-------------+-----+---------+--------------+--------+
  Total stock value: Rs 141,030.00
```

Figure 3. Inventory view (option 2).

6.3 Reorder alerts

```
  Reorder alerts
  NORMAL: quantity <= reorder level  |  URGENT: quantity is 0 or <= 50% of reorder level
+------+---------------------+-----+---------+-----------+--------+
| ID   | Name                | Qty | Reorder | Shortfall | Alert  |
+------+---------------------+-----+---------+-----------+--------+
| P008 | Desk Lamp           |   0 |       6 |         6 | URGENT |
| P003 | Mechanical Keyboard |   5 |      10 |         5 | URGENT |
| P002 | USB-C Cable         |  25 |      40 |        15 | NORMAL |
| P011 | Whiteboard Marker   |  55 |      60 |         5 | NORMAL |
| P004 | A4 Notebook         |  50 |      50 |         0 | NORMAL |
| P006 | Stapler             |   8 |       8 |         0 | NORMAL |
| P007 | Office Chair        |   5 |       5 |         0 | NORMAL |
+------+---------------------+-----+---------+-----------+--------+
```

Figure 4. Reorder alerts (option 7). Shortfall is reorder level minus quantity.

6.4 NumPy summary and category report

```
  NumPy summary (units per product)
+-----------------------+---------------+
| Metric                |         Value |
+-----------------------+---------------+
| Products              |            12 |
| Total units           |           303 |
| Mean units            |         25.25 |
| Median units          |          24.0 |
| Minimum units         |             0 |
| Maximum units         |            55 |
| Total inventory value | Rs 141,030.00 |
+-----------------------+---------------+
```

```
  Category report (Pandas)
+-------------+----------+-------+--------------+-----------------+
| Category    | Products | Units |  Stock value | Low-stock items |
+-------------+----------+-------+--------------+-----------------+
| Electronics |        5 |   105 | Rs 89,245.00 |               2 |
| Furniture   |        2 |     5 | Rs 27,495.00 |               2 |
| Home        |        1 |    35 | Rs 12,215.00 |               0 |
| Stationery  |        4 |   158 | Rs 12,075.00 |               3 |
+-------------+----------+-------+--------------+-----------------+
```

Figure 5. Options 9 and 10.

6.5 Monthly report and transaction history

```
  Monthly report
+---------+----------+-----------+---------------+---------------+
| Month   | Units in | Units out |      Value in |     Value out |
+---------+----------+-----------+---------------+---------------+
| 2026-07 |      845 |         0 | Rs 370,965.00 |       Rs 0.00 |
| 2026-08 |        0 |        15 |       Rs 0.00 |  Rs 11,985.00 |
| 2026-09 |        0 |       235 |       Rs 0.00 |  Rs 73,675.00 |
| 2026-10 |        0 |       292 |       Rs 0.00 | Rs 144,275.00 |
+---------+----------+-----------+---------------+---------------+
```

```
  Transaction history (8 of 39)
+---------+---------------------+---------+------+-----+-------------+--------------+
| Txn ID  | Timestamp           | Product | Type | Qty |  Unit price |        Value |
+---------+---------------------+---------+------+-----+-------------+--------------+
| T000039 | 2026-10-28 11:00:00 | P011    | OUT  |  25 |    Rs 45.00 |  Rs 1,125.00 |
| T000038 | 2026-10-27 11:00:00 | P008    | OUT  |  12 |   Rs 899.00 | Rs 10,788.00 |
| T000037 | 2026-10-26 11:00:00 | P002    | OUT  |  20 |   Rs 249.00 |  Rs 4,980.00 |
| T000036 | 2026-10-24 11:00:00 | P004    | OUT  |  50 |    Rs 60.00 |  Rs 3,000.00 |
| T000035 | 2026-10-23 11:00:00 | P006    | OUT  |  12 |   Rs 150.00 |  Rs 1,800.00 |
| T000034 | 2026-10-21 11:00:00 | P007    | OUT  |   6 | Rs 5,499.00 | Rs 32,994.00 |
| T000033 | 2026-10-20 11:00:00 | P001    | OUT  |  12 |   Rs 799.00 |  Rs 9,588.00 |
| T000032 | 2026-10-17 11:00:00 | P005    | OUT  |  15 |   Rs 120.00 |  Rs 1,800.00 |
+---------+---------------------+---------+------+-----+-------------+--------------+
```

Figure 6. Options 12 and 11 (latest 8 transactions).

6.6 Charts

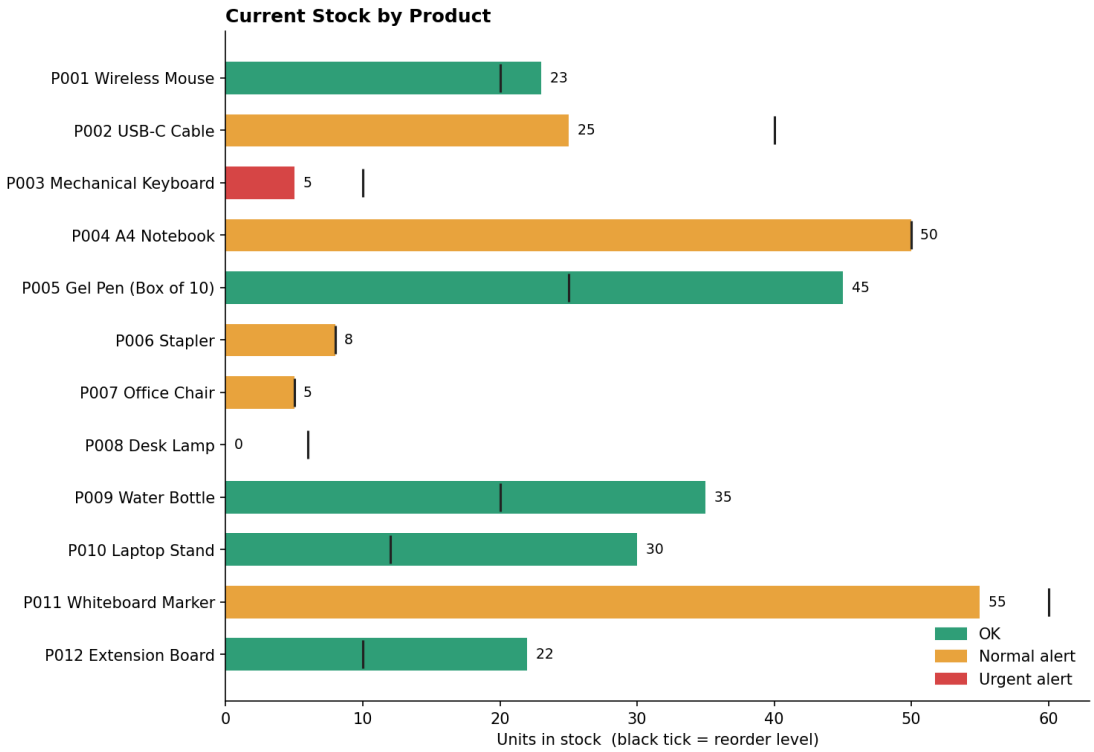


Figure 7. Stock by product. Green = no alert, amber = normal alert, red = urgent; the black tick marks each reorder level.

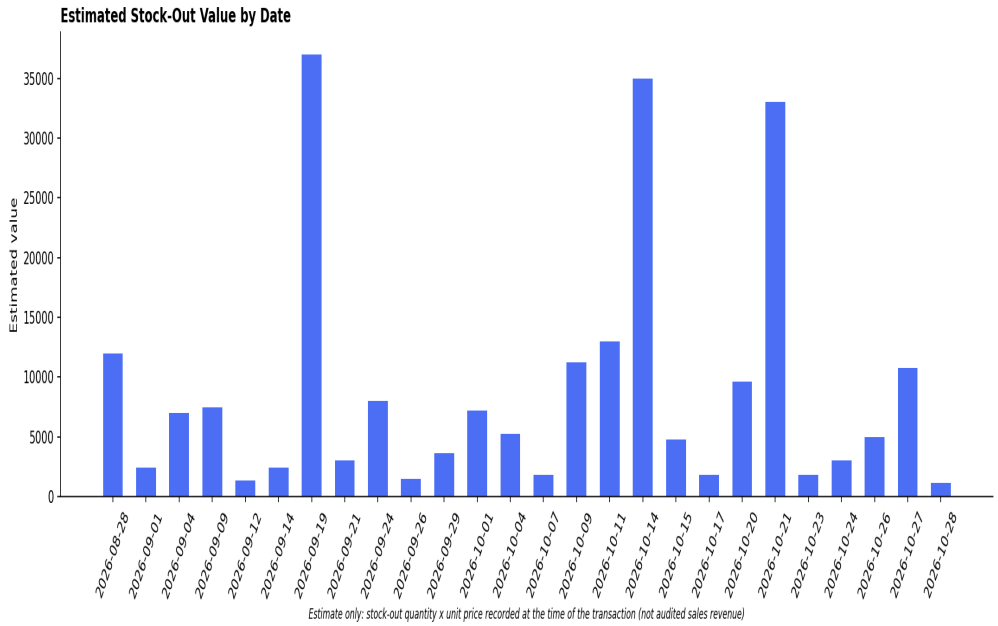


Figure 8. Estimated stock-out value by date. It is quantity x the price recorded at the time, not audited sales revenue.

6.7 Discussion

The alert rules correctly separate items that are merely at or below the reorder level from items that are empty or nearly empty. Because stock-out and stock-in always go through the same validated methods, the quantity of every product equals the net of its logged transactions; this property is checked by an automated test on the demo data. The sales chart cannot capture discounts, returns or price changes, which is why it is labelled as an estimate.

7. Testing and Validation

The file `tests/test_inventory_tracker.py` contains **51** tests written with Python's `unittest`. All tests use temporary directories, so real data is never changed. When this report was prepared, running `python -m unittest discover -s tests` reported **51 tests, all passed** (Python 3.12, NumPy 2.4, Pandas 3.0, Matplotlib 3.10). During development two failures were found; both were mistakes in the tests' expected values and were corrected.

Test area	Expected behaviour	Result
Product validation	Blank, malformed, negative or non-numeric fields rejected; text normalised	Passed (automated)
Duplicate product IDs	Rejected, case-insensitive; no partial record created	Passed (automated)
Stock-in / stock-out	Quantity and value correct; transaction recorded	Passed (automated)
Insufficient stock	Rejected; stock and log unchanged; exact full quantity allowed	Passed (automated)
Reorder boundaries	11/10 none, 10/10 normal, 5/10 urgent, 0 urgent, 0/0 urgent, 1/0 none	Passed (automated)
Valuation and rounding	19.99 x 3 = 59.97; totals agree with NumPy	Passed (automated)
CSV save and reload	Round trip identical; next transaction ID continues	Passed (automated)
Corrupt or missing files	Bad rows skipped with warnings; originals preserved; missing files give empty inventory	Passed (automated)
Permission and disk errors	StorageError raised; previous files intact (simulated)	Passed (automated)
JSON export	File parses with json.load; counts correct; empty export valid	Passed (automated)
Empty reports and charts	No crash; friendly message; no chart file created	Passed (automated)
Invalid menu and prompt input	Error message and re-prompt; 'cancel' returns to menu	Passed (automated)
Demo data	Consistent history; existing IDs never overwritten	Passed (automated)
Interactive chart window	Opens when 'y' is chosen	Manual check needed
Terminal appearance (colour, box characters)	Readable in the presentation computer's terminal	Manual check needed

Passing tests is evidence of correct behaviour in the situations tested; it does not prove that the program contains no bugs. Items marked 'Manual check needed' were not covered by automated tests and should be exercised on the computer used for the viva.

8. Limitations and Future Scope

Limitations:

- Single-user console program with no login or access control; CSV files are not a multi-user database.
- The sales chart is an estimate and ignores discounts, returns and price history.
- Each product has one unit price; there are no suppliers, batches, expiry dates or barcodes.
- Demonstration data is synthetic; alert thresholds (100% and 50% of the reorder level) are a design choice, not an industry standard.

Future scope: SQLite storage, a graphical or web interface, supplier purchase orders, suggested reorder quantities, user accounts, barcode scanning and price history.

9. Conclusion

The project delivers a complete, modular inventory tracker in Python. It validates input before every change, keeps stock and transaction history consistent, warns about low stock with documented thresholds, and supports analysis with NumPy, Pandas and Matplotlib. Safe saving and automated tests increase confidence in its reliability, while the remaining manual checks and stated limitations are identified openly.

10. References

Python Software Foundation. *Python 3 Documentation*. https://docs.python.org/3/

NumPy Developers. *NumPy Documentation*. https://numpy.org/doc/

Pandas Development Team. *pandas Documentation*. https://pandas.pydata.org/docs/

Matplotlib Development Team. *Matplotlib Documentation*. https://matplotlib.org/stable/

Python Software Foundation. *unittest - Unit testing framework*. https://docs.python.org/3/library/unittest.html

11. Appendix: Data Formats and Execution

Sample products.csv

```
product_id,name,category,unit_price,quantity,reorder_level
P001,Wireless Mouse,Electronics,799.0,23,20
P002,USB-C Cable,Electronics,249.0,25,40
```

Sample transactions.csv

```
transaction_id,timestamp,product_id,type,quantity,unit_price,value
T000001,2026-08-01 10:00:00,P001,IN,60,799.0,47940.0
```

Install and run

```
pip install -r requirements.txt
python inventory_tracker.py
python -m unittest discover -s tests -v
```

Project files: inventory_tracker.py, requirements.txt, README.md, tests/test_inventory_tracker.py, docs/sample_output.txt. The README contains a step-by-step manual test of every menu option.